Guia C++

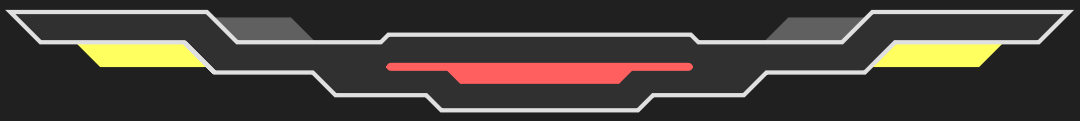
Per

Martí Figverola Giner



en procés

Continguts



1 Fonaments **3**
- 1.1 Diferències entre C i C++ . . . 3
- 1.2 Estructura . . . 3
- 1.3 Sintaxis . . . 4
- 1.4 Comentaris . . . 5
- 1.5 Capçaleres . . . 5
- 1.6 Arxius . . . 5
- 1.7 *Keywords* . . . 5

2 Dades **7**
- 2.1 Representació de Dades . . . 7
  - 2.1.1 Identificadors . . . 7
  - 2.1.2 Variables . . . 7
  - 2.1.3 Inicialització . . . 8
  - 2.1.4 Constants . . . 8
  - 2.1.5 Literals . . . 8
- 2.2 Tipus de Dades . . . 11
  - 2.2.1 Modifcadros . . . 11
- 2.3 Operadors . . . 11
  - 2.3.1 Operadors Aritmètics . . . 11
  - 2.3.2 Operadors d'Increment i Decrement . . . 12
  - 2.3.3 Operadors Relacionals . . . 13
  - 2.3.4 Operadors Lògics . . . 13
  - 2.3.5 Operadors Bit a Bit . . . 13
  - 2.3.6 Altres Operadors . . . 13
  - 2.3.7 Prioritat . . . 13
  - 2.3.8 Operadors d'Asignació . . . 13
- 2.4 *Arrays* o Llista? . . . 14
  - 2.4.1 Structs? . . . 14

3 Estructures de Control **15**
- 3.1 If . . . 15
- 3.2 case . . . 15
- 3.3 Funcions . . . 15

4 Pointers referencia **16**

5 Clases i Templates ?? **17**

6 Llibreria Estàndar **18**

7 Sintaxis i Conveció (resum) **19**

tipus var poo templates long long int static const structs enum

Fonaments



1.1 Diferències entre C i C++

Abans de C++ existia únicament el C, orientat a ser un llenguatge minimalístic, fàcil de compilar, eficient i ràpid. El punt més fort d'aquest llenguatge és que el programador té molt control sobre el programa.

Uns anys després es va desenvolupar C++, que pretenia ser una extensió del llenguatge original amb moltes més característiques, en què destaca la programació orientada a objectes.

També existeixen noves versions de C++, com les C++11, C++14, C++17, C++20, C++23, actualment la base es considera C++11, on es van implementar un gran nombre de noves característiques.

Aquests llenguatges són de compilació, és a dir, que primer s'escriu el codi, es compila, i es crea un arxiu `.exe`. Això té la ventatge que tarda una mica en compilar, però un cop compilat el programa s'executa molt ràpid.

1.2 Estructura

L'estructura principal tracta d'una funció que s'ha d'anomenar `main()`, de la següent forma:

```
int main()
{
    return 0;
}
```

Nota que a partir de C++11 no es necesari posar `return 0` [1], tot i que per coherència és recomenable posar-lo.

Normalment s'inclouen llibreries per poder fer més funcions, com per interactuar amb l'usuari per teclat. Això s'aconsegueix amb la llibreria `<iostream>`.

Amb aquesta llibreria bàsica es pot rebre arxius escrits amb `cin >> x`, en que `x` és una variable on anirà el valor escrit, `cout << x` escriurà `x` per pantalla i `endl` trenca la línea.

[1] Es podria retornar qualsevol altre nombre `int`, però per convenció es posa el 0.

```cpp
#include <iostream>

int main()
{
    std::cout << "Hola Món" << std::endl;
    return 0;
}
```

Al usar aquesta llibreria, hem d'escriure davant de cada variable que aquesta conté `std::`, però normalment per programes senzills ens podem estalviar això usant `using namespace std`.

```cpp
#include <iostream>
using namespace std;

int main()
{
    cout << "Hola Món" << endl;
    return 0;
}
```

És recomenable no usar `using namespace std` per programes més grans, ja que la llibreria que hem inclos conté molts noms que no podem repetir, excepte si afegim `std::`.

1.3 Sintaxis

La sintaxis de C++ permet usar els espais lliurament[2] per poder fer el codi més compacte visualment.

Les claus (`{}`) fan funció de parèntesis de codi, i els punts i coma (`;`) marquen el final d'una línea. Per tant si es posen varis punts i coma seguits o claus innecesàries el codi funciona bé igualment. El següent codi és igual de correcte:

```cpp
#include <iostream>
using namespace std; int main()
{ cout << "Hola Món" << endl;{return 0;;;}}
```

[2] Excepte després de les paraules clau (marcades en vermell) o clases.

Cal destacar que les capçaleres amb `#` no permeten tenir res al costat, necesàriament s'ha de trencar de línea.

1.4 Comentaris

Per comentar parts del codi sense que aquest text afecti el programa, es pot fer de dues formes diferents, d'una linea o multilinea.

```
#include <iostream> // llibreria bàsica per mostra text per pantalla

int main() // funció principal
{
    /*
    comentari multilinea per exemplificar com funcionen
    els comentaris en c++, aquest tipus de comentari va
    molt bé si es busca inhabilitar una part del codi temporalment
    */

    return 0;
}
```

1.5 Capçaleres

1.6 Arxius

1.7 *Keywords*

Les *keywords* són paraules predefinides amb un significat especial que serveixen per indicar accions que ha d'interpretar el compilador.

`alignas` (C++11)	`alignof` (C++11)	`and`	`and_eq`
`asm`	`atomic_cancel`	`atomic_commit`	`atomic_noexcept`
`auto`	`bitand`	`bitor`	`bool`
`break`	`case`	`catch`	`char`
`char8_t` (C++20)	`char16_t` (C++11)	`char32_t` (C++11)	`class`
`compl`	`concept` (C++20)	`const`	`consteval` (C++20)

constexpr (C++11)	constinit (C++20)	const_cast	continue
co_await (C++20)	co_return (C++20)	co_yield (C++20)	decltype (C++11)
default	delete	do	double
dynamic_cast	else	enum	explicit
export	extern	false	float
for	friend	goto	if
inline	int	long	mutable
namespace	new	noexcept (C++11)	not
not_eq	nullptr (C++11)	operator	or
or_eq	private	protected	public
reflexpr	register	reinterpret_cast	requires (C++20)
return	short	signed	sizeof
static	static_assert (C++11)	static_cast	struct
switch	synchronized	template	this
thread_local (C++11)	throw	true	try
typedef	typeid	typename	union
unsigned	using	virtual	void
volatile	wchar_t	while	xor
xor_eq			

Dades



2.1 Representació de Dades

Existeixen tres formes de represenació de dades, les variables, les constants i els literals. Totes aquestes representacions poden ser tipus de dades diferents.

2.1.1 Identificadors

Els indentificadors són els noms únics que es posen a les dates, ja siguin variables o constants. El nom dels indentificadors seguieixen unes certes normes perquè el compilador pugui executar-se i no confondre's amb *keywords*.

I. Es poden composar per lletres, dígits i la barra baixa.

II. No pot començar per un dígit.

III. Es diferencien les majúscules i les minúscules

IV. No hi ha límit de caràcters.

V. No han de tenir nom de cap *keyword* § 1.7.

2.1.2 Variables

Una variable és un espai de memòria on té un valor guardat que pot variar a través de les instruccions del programa.

Aquesta variable primer s'ha de declarar amb el tipus de data que vols que sigui i seguit d'un espai l'identificador. També es poden declarar múltiples variables del mateix tipus en una sola línea separades per comes. Després de declarar la variable es pot modificar el seu valor amb l'operador d'assignació `=` sense haver d'escriure de nou el tipus de variable.

```
int a;          // declaració de "a"
int b, c;       // declaració de "b" i "c"

a = 1;
b = 2, c = 3;   // ERROR, es necesita una altra línea de codi
```

En cada variable es pot donar el valor desitjat en la mateixa línea que es declara, inicialitzar.

2.1.3 Inicialització

Hi ha 4 formes de fer-ho: inicialització de còpia, inicialització directe, inicialització directa de llista i inicialització de còpia de llista. Es poden inicialitzar múltiples variables del mateix tipus en la mateixa línea amb qualsevol mètode d'inicialització intercanviadament.

```
int a = 1;      // inicialització de còpia
int b(2);       // inicialització directe
int c { 3 };    // inicialització directa de llista
int d = {4};    // inicialització de còpia de llista
```

La diferència principal entre aquestes inicialitzacions, és que la de còpia és una mica més lenta i que la de llista no permet valors que no siguin del tipus indicat.

```
int a = 4.5;    // a = 4, una mica lent
int b(4.5);     // b = 4, una mica més ràpid
int c { 4.5 };  // ERROR, no és un "int"
```

2.1.4 Constants

Una constant és un espai de memòria on té un valor guardat que no pot variar a través de les instruccions del programa.

Aquesta constant primer s'ha de declarar exactament com una variable però afegint davant `const`. El que és important recalcar és que obligatòriament s'ha de inicialitzar, perquè el valor no es podrà modificar.

```
const int a { 0 };              // inicialització de "a"
const int b { 2 }, c { 3 };     // inicialització de "b" i "c"

const int d;                    // ERROR
a = 3;                          // ERROR
```

2.1.5 Literals

Els literals són sempre constants d'un tipus concret sense cap identificador, per exemple un literal és un dígit en sí mateix. Depenent de la notació, aquests poden ser, enters, punt flotant, caracters, cadenes, booleans, punters i personalitzats.

Enters

Els enters són simplement una succesió de números que es poden representar de forma decimal, binaria (C++14), octal i hexadecimal. Per marcar que són negatius s'afegiex `-` abans del nombre.

```
14;         // decimal                  a = 14
-0b10;      // binari       ->   0b     b = -2
010;        // octal        ->   0      c = 8
-0xB;       // hexadecimal  ->   0x     d = -11
```

També es poden aplicar modificadors de tipus a mode de sufix. Aquests poden ser `unsigned` amb `u`, `long` amb `l` i `long long` amb `ll` [1].

A més també es poden combinar per formar `unsigned long` i `unsigned long long` de les següents maneres:

```
75u;        // unsigned             (també U)
75l;        // long                 (també L)
75ll;       // long long            (també LL)
75ul;       // unsigned long        (també UL, lU, Lu...)
75ull;      // unsigned long long   (també llu, Ull, LLU...)
```

Punt Flotant

Els nombres de punt flotant poden ser `double` o `float`, i permeten usar nombres amb dècimes. Per denotar que parlem de punts flotants s'ha d'afegir al final del nombre un punt a mode de coma per marcar les dècimes.

Es poden escriure de forma decimal amb notació científica i en hexadecimal (C++17) obligatòriament amb potència. Per la notació científica s'usa `e` i per la potència `p` [2].

```
3.0;            // decimal          ->  3
-13e-2;         // decimal exponent -> -0.13
0x1.4Ep3;       // hexadecimal      -> 10.4375
// 1.4Ep3 = ( 1 + 4/16 + 14/256 ) * 2^3
```

[1] També es poden escriure en majúsucles i combinar-se entre sí, tot i que amb `long long` ha de ser `ll` o `LL`, no és vàlid `lL`.
[2] Igual que amb els enters, aquests poden ser en majúscules o en minúscula.

També es poden aplicar els modificadors de tipus a mode de sufix. Aquests poden ser `long double` amb `l` i `float` amb `f`[3]. Es interesant usar el modificador `float` perquè per defecte ho expresa com un `double` si no s'especifica.

Caracters i Cadenes

Els caràcters són una unitat textual que es representa entre cometes simple (`'`). Les cadenes són una llista de caràcters que es representa entre cometes dobles (`"`).

Existeixen caràcters especials de dos unitats textuals (que es representen entre cometes simples) per fer una acció o representar caràcters especials.

Codi	Descripció
`\n`	Crea una línea nova
`\t`	Tabula horitzontalment
`\v`	Tabula verticalment només en alguns entorns, obsolet
`\b`	Retroces, elimina l'anterior caràcter
`\r`	Retroces de línea, elimina la línea escrita
`\f`	Salt de pàgina, només en alguns entorns, obsolet
`\a`	Alerta sonora, només en alguns entorns, obsolet
`\0`	Caràcter nul, normalment per indicar el final d'una cadena
`\'`	Cometa simple
`\"`	Cometes dobles
`\?`	Símbol d'interrogació
`\\`	Barra invertida

Els caràcters es guarden de forma númerica, per tant es poden indicar els caràcters de forma númerica sense les cometes si es sap que es guardar en un tipus `char`. A més es pot representar numèricament sense la necesitat de guardar-ho en un `char`, es pot fer entre cometes amb forma octal i hexadecimal.

```
char a { 65 };      // A, en decimal, s'ha d'especificar que és char
char b { '\101' };  // A, en octal, s'introdueix per \
char c { '\x4A' };  // J, en hexadecimal, s'introdueix per \x
```

[3] També es poden escriure en majúsucles.

Aquests codis números segueixen la *taula ASCII*, tot i que la taula extensa pot no funcionar.

També es poden aplicar prefixos per determinar el tipus de caràcter que es busca, tot i que aquests prefixes distingeixen majúscules i minúscules.

u char???

Concatenar strings

Keywords

Només hi ha 3 *keywords* que són literals, dos booleanes (`true` i `false`) i una de punter (`nullptr`).

Personalitzats

2.2 Tipus de Dades

auto

int pot ser en binario 0b en c++14 y hex amb 0x en totes y octal amb 0 en totes les versions, tot ique es guarda com un enter normal. tambe la notacio cientifica 0.22E-5

auto decltype

75 // int 75u // unsigned int 75l // long 75ul // unsigned long 75lu // unsigned long

els chars i els bools es guarden com a numeros y es poden posar com a numeros (crec)

2.2.1 Modifcadros

(long long) static

2.3 Operadors

2.3.1 Operadors Aritmètics

Els operadors aritmètics són aquells que retornen un valor numèric. Aquests operadors interactúen imitant les regles matemàtiques del ordre del operands i parèntesis.

Operador	Símbol	Forma	Descripció
Suma unària	`+`	`+x`	Valor de `x`
Resta unària	`-`	`-x`	Valor invertit de `x`
Suma	`+`	`x + y`	Suma de `x` i `y`
Resta	`-`	`x - y`	Resta de `x` i `y`
Multiplicació	`*`	`x * y`	Multiplicació de `x` i `y`
Divisió	`/`	`x / y`	Divisió de `x` i `y`
Mòdul	`%`	`x % y`	Restant de la divisió de `x` i `y`

Si en una operació binària un dels dos operands es un punt flotant la operació considerarà que el enter és un punt flotant. Concretament en la divisió, si els dos valors són enters i el resultat hauría de donar un valor amb decimals, només es retornarà la part entera.

Per altra banda l'operador `%` resulta molt útil per fer operacions modulars amb combinació de la divisió entera, per exemple alhora de comptar dies a partir d'hores.

```
int horesTotals { 73 };
int dies { horesTotals / 24 };  // retorna 3 en comptes de 3.04...
int hores { horesTotals % 24 }; // retorna 1, 3 dies i 1 hora
```

2.3.2 Operadors d'Increment i Decrement

Aquest operadors són comparteixen símbols però no forma. Serveixen per incrementar o decrementar amb diferent prioritat.

Operador	Símbol	Forma	Descripció
Increment (prefix)	`++`	`++x`	Incrementa `x`, retorna `x`
Decrement (prefix)	`--`	`--x`	Decrementa `x`, retorna `x`
Increment (sufix)	`++`	`x++`	Copia `x`, incrementa `x`, retorna la copia
Decrement (sufix)	`--`	`x--`	Copia `x`, decrementa `x`, retorna la copia

Es recomana que s'usi el prefixe quan no es busca jugar amb la prioritat. Els de sufix poden ser útils alhora d'interactuar amb una posició d'una llista i colocar-se en la següent posició, per exemple `llista[i++]`, en la següent iteració `i` haurà incrementat a la següent posició.

2.3.3 Operadors Relacionals

Els operadors relacionals comparen dos valors i retornen un booleà, `true` si es compleix i `false` si no es compleix.

Operador	Símbol	Forma	Descripció
Igualtat	`==`	`x == y`	Si `x` és igual a `y`
Desigualtat	`!=`	`x != y`	Si `x` és diferent a `y`
Major que	`>`	`x > y`	Si `x` és major a `y`
Menor que	`<`	`x < y`	Si `x` és menor a `y`
Major o igual que	`>=`	`x >= y`	Si `x` és major o igual a `y`
Menor o igual que	`<=`	`x <= y`	Si `x` és menor o igual a `y`

2.3.4 Operadors Lògics

Aquests operadors són equivalents a les 3 portes lògiques fonamentals. Són útils quan es combinen amb els operadors relacionals ja que comparen booleans.

Operador	Símbol	Forma	Símbol Alternatiu	Forma Alternativa
`NOT` lògic	`!`	`!x`	`not`	`not x`
`AND` lògic	`&&`	`x && y`	`and`	`x and y`
`OR` lògic	`||`	`x || y`	`or`	`x or y`

2.3.5 Operadors Bit a Bit

Aquests operadors tenen en compte la forma binària en que es guarda en memòria per fer els càlculs bit per bit.

2.3.6 Altres Operadors

? , sizeof -> [] :: .* ->*

2.3.7 Prioritat

2.3.8 Operadors d'Asignació

Els operadors d'assignació són útils per simplificar petities operacions amb la mateixa variable.

Operador	Símbol	Forma	Descripció
Assignació	`=`	`x = y`	Assigna a `x` el valor `y`
Assignació suma	`+=`	`x += y`	Assigna a `x` el valor `x + y`
Assignació resta	`-=`	`x -= y`	Assigna a `x` el valor `x - y`
Assignació multiplicació	`*=`	`x *= y`	Assigna a `x` el valor `x * y`
Assignació divisió	`/=`	`x /= y`	Assigna a `x` el valor `x / y`
Assignació mòdul	`%=`	`x %= y`	Assigna a `x` el valor `x % y`
Assignació divisió	`&`	`x &= y`	Assigna a `x` el valor `x & y`
Assignació divisió	`|`	`x |= y`	Assigna a `x` el valor `x | y`
Assignació divisió	`^`	`x ^= y`	Assigna a `x` el valor `x ^ y`
Assignació divisió	`<<`	`x <<= y`	Assigna a `x` el valor `x << y`
Assignació divisió	`>>`	`x >>= y`	Assigna a `x` el valor `x >> y`

Cal destacar que si s'assigna un tipus de dada que no és el que requereix la variable el compilador intentarà interpretar-lo com el que ha estat declarat, tot i que algunes combinacions poden donar error.

```
int a;
a = 3.5;        // a = 3
a = true;       // a = 1
a = 'A';        // a = 65
```

2.4 *Arrays* o Llista?

hi ha diccionari?

2.4.1 Structs?

Estructures de Control



3.1 If

3.2 case

3.3 Funcions

Pointers referencia

4

Clases i Templates ??



Llibreria Estàndar



string, vector, cout...

Sintaxis i Conveció (resum)

7

```
int main()
{
    const int CONSTANT = 3;
    int variableEnCamelCase;


    for (int i = 0; i < n; i++)
        if (x == x)
            return 0;

    class Clase
    public:
        constructors + destructors + constructor de copia

        getters + setters
        Clase<T1, T2, valor> getValor1() const { return m_valor1; }

        sobrecarrega d'operadors + operador d'asignació

        friend %&ostream operator << ¿?

        metodes normals
    private:
        atributs (potser en m_)

        metodes
}
```

(sense corxets) for (int i = 0; i < n; i++) if (x == x) return 0;

class Clase public: constructors + destructors + constructor de copia

getters + setters

sobrecarrega d'operadors + operador d'asignació

friend

metodes normals private: atributs

metodes

Written by Martí Figuerola

'If you want something done right, do it right'



Started in Septembre 2024

